



Lab 3: Remote Repositories & GitHub Integration

Setup

1. Make sure you have Git installed and configured (see Lab 1).
2. Create a GitHub account at github.com , or sign in with your existing account.
3. Create a new **public repository** on GitHub (do not initialize it with a README).

You should see the following:

1

2

```
...or create a new repository on the command line

echo "# Test" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/RandomCatGit/Test.git
git push -u origin main

...or push an existing repository from the command line

git remote add origin https://github.com/RandomCatGit/Test.git
git branch -M main
git push -u origin main

...or import code from another repository
You can initialize this repository with code from a Subversion, Mercurial, or TFS project.
Import code
```

If you have an existing repository, use command in the second section.

Otherwise, if you want to create a new folder and start from scratch use commands in the first section.



Exercise 1: Push to Remote Repository

1. Verify your remote using **git remote -v**
2. Push your commits to GitHub.
3. Open your repository in a browser and confirm your files appear on GitHub.
4. Make a new local commit (edit README.md and commit).
5. Push again to see the new commit.

Deliverables

Output of git push command showing the upload to GitHub.

Exercise 2: Fetch VS Pull

This exercise will help you visualize the difference between git fetch and git pull.

1. On GitHub, edit README.md directly in the web interface (add a new line).
2. Back in your local repo, **fetch** the new changes
Observe that the change is downloaded but not merged).
3. Check your local branch, it should be unchanged.
4. Now **pull** the changes.
5. Check commit history to see the new commit from GitHub.

Deliverables

Output of git fetch and git pull commands.



Exercise 3: Using .gitignore

1. Create a file named **.gitignore** in your working directory
2. Add the following lines:
***.log**
node_modules/
.env
3. Create a file named **test.log** and confirm it doesn't appear in git status.
If you try to add and commit this file, it won't be seen. This ensures sensitive or unnecessary files aren't tracked.
4. You can verify the ignored files using: **git status --ignored**

Deliverables

Output of git fetch and git pull commands.



Exercise 4: Cloning a public repository using HTTPS (Optional)

This exercise demonstrates how to clone a **public** GitHub repository using the **HTTPS**. For **private repositories**, you will need authentication (using a Personal Access Token or SSH key), we'll cover that in the next lab.

1. Visit your GitHub repository page → Click **"Code"** → Observe the two URLs:
HTTPS: `https://github.com/username/repo.git`
SSH: `git@github.com:username/repo.git`
2. Create a new folder on your local machine and run: **`git clone <url>`**
3. Observe that HTTPS allows you to clone your **public** repository without authentication.
4. Try cloning a **private** repository or pushing to a **public repository that isn't yours**. Git will ask for authentication.
We will explore how to handle this securely in the next lab.

Exercise 5: Creating a Simple Pull Request (Optional)

1. Create a new branch locally.
2. Add a new file `main.cpp` (or any file you want), commit, and push it.
*To push a local branch use **`git push origin <branch-name>`***
3. On GitHub, open a **Pull Request** from your new branch into `main`.
4. Add a short title and description, then click **Create Pull Request**.
5. Merge it through the GitHub interface.